SCHOOL OF TECHNOLOGY	

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

RESTAURANT MANAGEMENT SYSTEM

Program	B-TECH
Section	AIML-RHINOS
Faculty	Prof.Sahithi
Date of Submission	
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102073	E.Nithya Lekhana	9390923235	nithya.erolla_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course.

This project provided an opportunity to connect database theory with application development. During the implementation, concepts such as relational schema design, entity-relationship modelling, normalization, primary and foreign keys, constraints, transactions, CRUD operations, database connectivity and reporting were applied to a restaurant-based operational scenario.

I am grateful to the course faculty for the academic guidance and feedback provided throughout the project. I also acknowledge the technical documentation that supported the implementation of MySQL, Python, Flask and database connectivity [1]–[4].

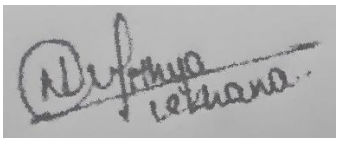
UNDERTAKING

I hereby declare that the project entitled **"Restaurant Management System"**, submitted as part of the Project-Based Learning activity for the Database Management Systems course, represents the project work submitted for academic evaluation.

The report has been prepared using the database schema, application source code, migration script, README documentation and testing files associated with the submitted project.

I undertake that the database structures, application workflow, SQL operations, constraints and implementation details presented in this report correspond to the submitted project. External technical concepts and documentation used during the development have been acknowledged through IEEE-style references and in-text citations.

Signature:



Name: E. Nithya Lekhana
Roll Number: 25WU0102073
Date: 05 October 2026

ABSTRACT

The Restaurant Management System is a database-driven web application developed to organize and manage information generated during restaurant operations. Restaurant activities such as customer registration, table allocation, reservations, menu management, order processing, kitchen operations, billing, discounts, and payments generate related data that must be stored consistently. The main objective of the project is to design and implement a centralized relational database and connect it with a web-based application for efficient restaurant management. The system uses MySQL as the database management system and Python with Flask for backend application development. The database is designed using an Entity-Relationship model and normalized relational schema with primary keys, foreign keys, unique constraints, check constraints, views, and validation triggers. The application provides modules for dashboard monitoring, customers, tables, reservations, menu items, orders, kitchen processing, billing, discounts, payments, and reports. CRUD operations and business-rule validations are implemented to maintain data consistency. The completed system demonstrates how DBMS concepts can be integrated with application development to provide structured data storage, controlled workflows, validation, and operational reporting for a restaurant environment.

Keywords: Restaurant Management System, Relational Database, MySQL, Flask, Normalization, SQL

.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	**Abstract**	4
2	**Chapter 1 – Problem Statement, Solution Strategy and ER Diagram**	10
2.1	Problem Statement and Objectives	10
2.2	Scope and Requirements	11
2.3	Solution Strategy	12
2.4	Entity-Relationship Diagram	14
3	**Chapter 2 – Logical Database Design**	16
3.1	Relational Schema	16
3.2	Keys and Constraints	17
3.3	Functional Dependencies	19
3.4	Normalization	20
3.5	Data Dictionary	22
3.6	Implementation of Logical Design: DDL and Sample Data	25
4	**Chapter 3 – Application Development**	26
4.1	Application Overview	26
4.2	Development Environment and Database Connectivity	27
4.3	Application Modules and Database Mapping	28
4.4	CRUD Operations	29
4.5	Business Rules and Validation	30
4.6	Reports and Database Operations	31

4.7	Application Screens	32
4.8	Testing and Results	35
5	**Chapter 4 – Conclusion**	37
5.1	Limitations and Future Enhancements	38
6	**Chapter 5 – References**	38
7	**Chapter 6 – Appendices**	39

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	11
Table 2	Proposed Solution Strategy	12
Table 3	Functional Requirements	13
Table 4	Entity and Relationship Summary	15
Table 5	Relational Schema	16
Table 6	Keys and Integrity Constraints	17
Table 7	Normalization Status up to 3NF	20
Table 8	Data Dictionary	22
Table 9	Database Implementation Environment	25
Table 10	Application Development Environment	27
Table 11	Application Module-to-Database Mapping	28
Table 12	CRUD Operations	29
Table 13	Business Rules, Validation and Error Handling	30
Table 14	Reports and Database Operations	31
Table 15	Test Cases	35
Table 16	Limitations and Future Enhancements	38

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Restaurant Management System	14
Figure 2	Restaurant Management System Dashboard	32
Figure 3	Reservation Management Screen	33
Figure 4	Order Management Screen	33
Figure 5	Billing and Payment Screen	34
Figure 6	Reports Screen	34

LIST OF ACRONYMS

Acronym	Expansion
DBMS	Database Management System
ERD	Entity-Relationship Diagram
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
REST	Representational State Transfer
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
UI	User Interface
PK	Primary Key
FK	Foreign Key
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Restaurant operations involve several interconnected activities such as customer management, table allocation, reservations, menu management, order processing, kitchen operations, billing, discounts, and payment processing. When these activities are maintained separately, it becomes difficult to maintain relationships between records, avoid duplicate information, and ensure that updates are applied consistently.

A relational database provides a structured approach for storing related information using tables, keys, constraints, and relationships. MySQL provides facilities for creating relational schemas, defining constraints, and executing SQL-based database operations [1].

The proposed Restaurant Management System addresses this problem by maintaining restaurant information in a centralized database. The application connects the database with a web-based interface so that restaurant operations can be performed through controlled application workflows.

The system also manages operational state changes. For example, reservations can be created and updated, orders can progress through service stages, kitchen tickets can be updated, bills can be generated, payments can be recorded, and tables can be released after completion of service.

1.2 Objectives

The major objectives of the project are:

1. To **design** a normalized relational database for restaurant operations.
2. To **implement** primary keys, foreign keys, unique constraints, check constraints, and other integrity rules.
3. To **develop** a Python-based backend application for interacting with the database [2].

4. To **use** Flask for backend routing and REST API services [3].
5. To **implement** customer, table, reservation, menu, order, kitchen, billing, discount, payment, and reporting modules.
6. To **implement** CRUD operations for major application entities.
7. To **enforce** important business rules and validation conditions.
8. To **generate** database-backed operational reports.
9. To **validate** important positive and negative application workflows.
10. To **demonstrate** practical integration between database design and web application development.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Area	Scope / Assumption
Customers	Customer information is maintained in the database.
Dining Areas	Restaurant areas are maintained and associated with restaurant tables.
Tables	Table number, capacity, and status are maintained.
Reservations	Reservations are linked to customers and restaurant tables.
Menu	Menu items and their prices and availability are maintained.
Waiters	Waiter information is maintained for restaurant operations.
Orders	Customer orders and ordered items are stored.
Kitchen	Kitchen tickets are used to track order preparation status.
Billing	Bills store order-related billing information.
Discounts	Discounts can be associated with billing operations.

Payments	Payments are recorded against bills.
Reports	Operational information is displayed through database queries and reporting views.
Authentication	Staff authentication is outside the current project scope.
External Payment Gateway	External payment gateway integration is outside the current scope.

The current implementation is intended as an academic project prototype rather than a complete production restaurant platform.

1.4 Proposed Solution Strategy

The proposed solution uses a centralized relational database and a web application layer.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users	Restaurant staff and authorized operational users
Database	MySQL 8.0
Backend	Python with Flask
Frontend	HTML, CSS and JavaScript
Database Connectivity	MySQL connectivity from the Python backend
Major Modules	Dashboard, customers, tables, reservations, menu, orders, kitchen, billing, discounts, payments and reports
Data Integrity	Primary keys, foreign keys, unique constraints, check constraints and validation triggers
Reporting	SQL queries and reporting views
Validation	Application-level and database-level validation

Main Workflow	Customer → Reservation/Table → Order → Kitchen → Bill → Payment

Proposed Workflow

1. Customer information is registered.
2. A suitable restaurant table is selected.
3. A reservation can be created for the customer.
4. Orders are created against the relevant customer/table.
5. Order items are stored separately.
6. Kitchen tickets track food preparation.
7. A bill is generated for the order.
8. Applicable discounts are validated.
9. Payment is recorded against the bill.
10. The table status can be updated after service completion.
11. Operational data is displayed through dashboards and reports.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	The system shall maintain customer information.	High
FR-02	The system shall maintain restaurant tables and capacity information.	High
FR-03	The system shall create and manage reservations.	High

FR-04	The system shall maintain menu items and prices.	High
FR-05	The system shall create and manage orders and order items.	High
FR-06	The system shall track kitchen ticket status.	High
FR-07	The system shall generate and manage bills.	High
FR-08	The system shall validate and apply discounts.	Medium
FR-09	The system shall record payments against bills.	High
FR-10	The system shall provide operational reports.	Medium
FR-11	The system shall prevent invalid reservations and payments.	High
FR-12	The system shall provide dashboard information for restaurant operations.	Medium

1.6 ENTITY-RELATIONSHIP DIAGRAM

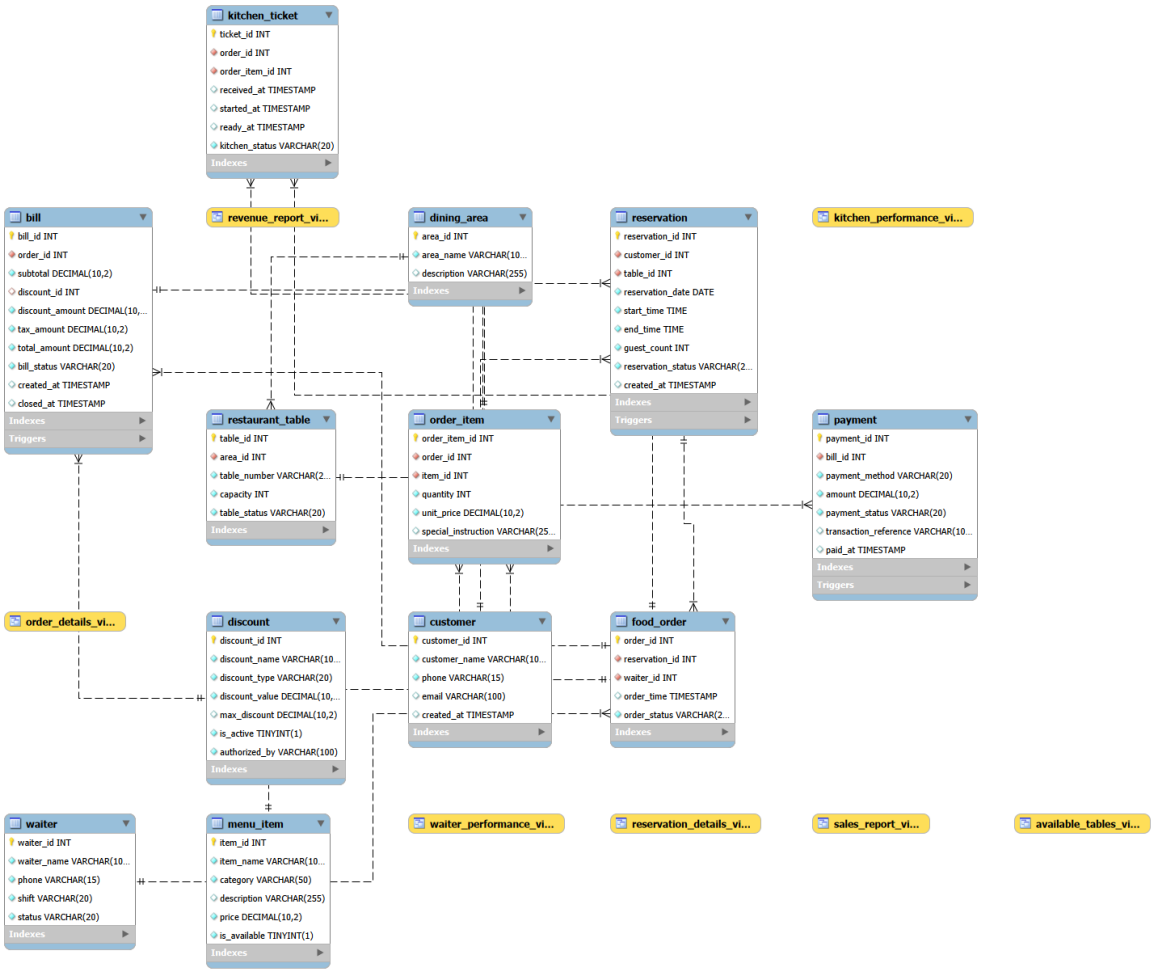


Figure 1. Entity-Relationship Diagram of the Restaurant Management System

The final implemented database contains the following **12 tables**:

1. dining_area
2. restaurant_table
3. customer
4. reservation
5. menu_item
6. waiter
7. food_order
8. order_item

9. kitchen_ticket
10. discount
11. bill
12. payment

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Dining Area	Area ID, area name	Stores restaurant areas.
Restaurant Table	Table ID, table number, capacity, status	Stores physical dining tables.
Dining Area → Restaurant Table	1 : N	One area can contain multiple tables.
Customer	Customer ID, name, phone, email	Stores customer information.
Customer → Reservation	1 : N	A customer can have multiple reservations.
Restaurant Table → Reservation	1 : N	A table can have multiple reservations at different times.
Menu Item	Menu Item ID, item name, price	Stores available food items.
Waiter	Waiter ID, waiter details	Stores waiter information.
Food Order	Order ID, customer/table references, status	Stores customer orders.

Food Order → Order Item	1 : N	One order can contain multiple items.
Menu Item → Order Item	1 : N	A menu item can appear in multiple order items.
Food Order → Kitchen Ticket	1 : N / operational relationship	Tracks kitchen processing.
Food Order → Bill	1 : 1 / controlled workflow	Billing is associated with an order.
Bill → Discount	Controlled relationship	Valid discounts can be applied to bills.
Bill → Payment	1 : N	Payment records are associated with bills.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model is converted into a relational schema in which entities are represented as tables and relationships are implemented using primary and foreign keys. MySQL 8.0 is used for the database implementation [1].

Table 5. Relational Schema

S.No.	Table Name	Primary Key	Purpose
1	dining_area	area_id	Stores restaurant dining areas.
2	restaurant_table	table_id	Stores restaurant table information.
3	customer	customer_id	Stores customer information.
4	reservation	reservation_id	Stores table reservations.

5	menu_item	menu_item_id	Stores food/menu information.
6	waiter	waiter_id	Stores waiter information.
7	food_order	order_id	Stores customer orders.
8	order_item	order_item_id	Stores individual items belonging to orders.
9	kitchen_ticket	ticket_id	Stores kitchen processing information.
10	discount	discount_id	Stores discount information.
11	bill	bill_id	Stores billing information.
12	payment	payment_id	Stores payment information.

2.2 Keys and Integrity Constraints

Table 6. Keys and Integrity Constraints

Table	Key / Constraint	Attribute(s)	Purpose
dining_area	Primary Key	area_id	Uniquely identifies each dining area.
restaurant_table	Primary Key	table_id	Uniquely identifies each table.
restaurant_table	Foreign Key	area_id	Connects table to dining area.
customer	Primary Key	customer_id	Uniquely identifies customers.
customer	Unique	Phone / Email where applicable	Prevents duplicate customer contact values.
reservation	Primary Key	reservation_id	Uniquely identifies reservations.

reservation	Foreign Keys	Customer/Table references	Maintains reservation relationships.
menu_item	Primary Key	menu_item_id	Uniquely identifies menu items.
waiter	Primary Key	waiter_id	Uniquely identifies waiters.
food_order	Primary Key	order_id	Uniquely identifies orders.
order_item	Primary Key	order_item_id	Uniquely identifies order items.
order_item	Foreign Keys	Order/Menu Item	Maintains order-item relationships.
kitchen_ticket	Primary Key	ticket_id	Uniquely identifies kitchen tickets.
discount	Primary Key	discount_id	Uniquely identifies discounts.
bill	Primary Key	bill_id	Uniquely identifies bills.
payment	Primary Key	payment_id	Uniquely identifies payments.
Various	NOT NULL	Required attributes	Prevents missing mandatory values.
Various	CHECK	Domain conditions	Prevents invalid values.

The database uses integrity constraints to reduce invalid and inconsistent records. MySQL supports the relational DDL and constraint mechanisms used to implement the schema [1].

2.3 Functional Dependencies

Functional dependencies describe how one attribute determines another attribute within a relation.

Important functional dependencies in the system include:

Customer

customer_id → customer_name, phone, email

A particular customer_id uniquely determines the corresponding customer information.

Dining Area

area_id → area_name

Restaurant Table

table_id → area_id, table_number, capacity, status

Menu Item

menu_item_id → item_name, price, availability

Reservation

reservation_id → customer_id, table_id, reservation_date, start_time, end_time, status

Food Order

order_id → customer_id, table_id, waiter_id, status

Order Item

order_item_id → order_id, menu_item_id, quantity

Kitchen Ticket

ticket_id → order_id, status

Bill

bill_id → order_id, subtotal, tax_amount, discount_amount, total_amount, status

Payment

payment_id → bill_id, amount, payment_method

These dependencies support the identification of candidate keys and the normalization of the database.

2.4 Normalization up to Third Normal Form

Normalization is used to reduce redundancy and avoid insertion, update, and deletion anomalies.

First Normal Form — 1NF

The database satisfies 1NF because attributes contain atomic values and repeating groups are not stored within individual fields.

For example, individual order items are stored in the order_item table rather than storing multiple menu items in a single column.

Second Normal Form — 2NF

The database satisfies 2NF because non-key attributes depend on the complete primary key. Separate entities such as customers, menu items, orders, and order items prevent partial dependency problems.

Third Normal Form — 3NF

The database satisfies 3NF because non-key attributes depend on the primary key and not on other non-key attributes.

Table 7. Normalization Status up to 3NF

Relation	1NF	2NF	3NF	Remarks
customer	Satisfied	Satisfied	Satisfied	Customer details depend on customer_id.
restaurant_table	Satisfied	Satisfied	Satisfied	Table attributes depend on table_id.
reservation	Satisfied	Satisfied	Satisfied	Reservation attributes depend on reservation_id.
menu_item	Satisfied	Satisfied	Satisfied	Menu attributes depend on menu_item_id.
food_order	Satisfied	Satisfied	Satisfied	Order attributes depend on order_id.

order_item	Satisfied	Satisfied	Satisfied	Item details are separated from order-level information.
bill	Satisfied	Satisfied	Satisfied	Billing attributes depend on bill_id.
payment	Satisfied	Satisfied	Satisfied	Payment attributes depend on payment_id.

Example of Normalization

An unnormalized order structure could contain:

Order(order_id, customer_name, customer_phone,

item_name, item_price, quantity)

This would duplicate customer and item information.

After normalization:

Customer(customer_id, customer_name, phone, email)

Food_Order(order_id, customer_id, table_id, waiter_id, status)

Menu_Item(menu_item_id, item_name, price)

Order_Item(order_item_id, order_id, menu_item_id, quantity)

This separation reduces duplication and improves consistency.

2.5 Data Dictionary

Table 8. Data Dictionary

Table	Important Attribute	Data Type	Key / Constraint	Description
dining_area	area_id	INT	PK	Unique dining area identifier

dining_area	area_name	VARCHAR	NOT NULL	Dining area name
restaurant_table	table_id	INT	PK	Unique table identifier
restaurant_table	table_number	INT/VARCHAR	Unique/Required	Table number
restaurant_table	capacity	INT	CHECK	Maximum number of guests
restaurant_table	status	VARCHAR	Required	Current table status
customer	customer_id	INT	PK	Unique customer identifier
customer	customer_name	VARCHAR	NOT NULL	Customer name
customer	phone	VARCHAR	Unique where applicable	Customer phone
customer	email	VARCHAR	Unique where applicable	Customer email
reservation	reservation_id	INT	PK	Unique reservation identifier
reservation	customer_id	INT	FK	Customer making reservation

reservation	table_id	INT	FK	Reserved table
menu_item	menu_item_id	INT	PK	Unique menu item identifier
menu_item	item_name	VARCHAR	NOT NULL	Food item name
menu_item	price	DECIMAL	CHECK	Item price
waiter	waiter_id	INT	PK	Unique waiter identifier
food_order	order_id	INT	PK	Unique order identifier
food_order	customer_id	INT	FK	Customer associated with order
food_order	table_id	INT	FK	Table associated with order
food_order	waiter_id	INT	FK	Waiter handling order
food_order	status	VARCHAR	Required	Order status
order_item	order_item_id	INT	PK	Unique order-item identifier

order_item	order_id	INT	FK	Related order
order_item	menu_item_id	INT	FK	Related menu item
order_item	quantity	INT	CHECK	Quantity ordered
kitchen_ticket	ticket_id	INT	PK	Unique kitchen ticket
kitchen_ticket	order_id	INT	FK	Related order
kitchen_ticket	status	VARCHAR	Required	Kitchen processing status
discount	discount_id	INT	PK	Unique discount
discount	code	VARCHAR	Required/Unique where applicable	Discount code
bill	bill_id	INT	PK	Unique bill
bill	order_id	INT	FK	Related order
bill	subtotal	DECIMAL	CHECK	Pre-discount amount
bill	tax_amount	DECIMAL	CHECK	Tax amount
bill	total_amount	DECIMAL	CHECK	Final bill amount

payment	payment_id	INT	PK	Unique payment
payment	bill_id	INT	FK	Related bill
payment	amount	DECIMAL	CHECK	Payment amount
payment	payment_method	VARCHAR	CHECK	Payment method

2.6 IMPLEMENTATION OF LOGICAL DESIGN: DDL AND SAMPLE DATA

The database is implemented using **MySQL 8.0** [1].

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL 8.0
Database Name	restaurant_management
Number of Operational Tables	**12**
Reporting Views	**7**
Stored Procedures	**0**
Validation Triggers	**6**
Backend Language	Python
Application Framework	Flask

Representative DDL

```
CREATE TABLE customer (

    customer_id INT NOT NULL AUTO_INCREMENT,
```

```
    customer_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) UNIQUE,
    email VARCHAR(100) UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (customer_id)
);
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Restaurant Management System is implemented as a web application that provides a user interface for restaurant operations. The frontend communicates with the Python Flask backend through API endpoints. Flask manages application routing and backend request handling [3]. Python is used for application logic and validation [2].

The backend communicates with the MySQL database to create, retrieve, update, and manage restaurant information. The application is therefore directly connected to the logical database design rather than relying on isolated static data.

The major application modules include:

1. Dashboard
2. Customers
3. Tables
4. Reservations
5. Menu
6. Orders

7. Kitchen
8. Billing and Payments
9. Reports

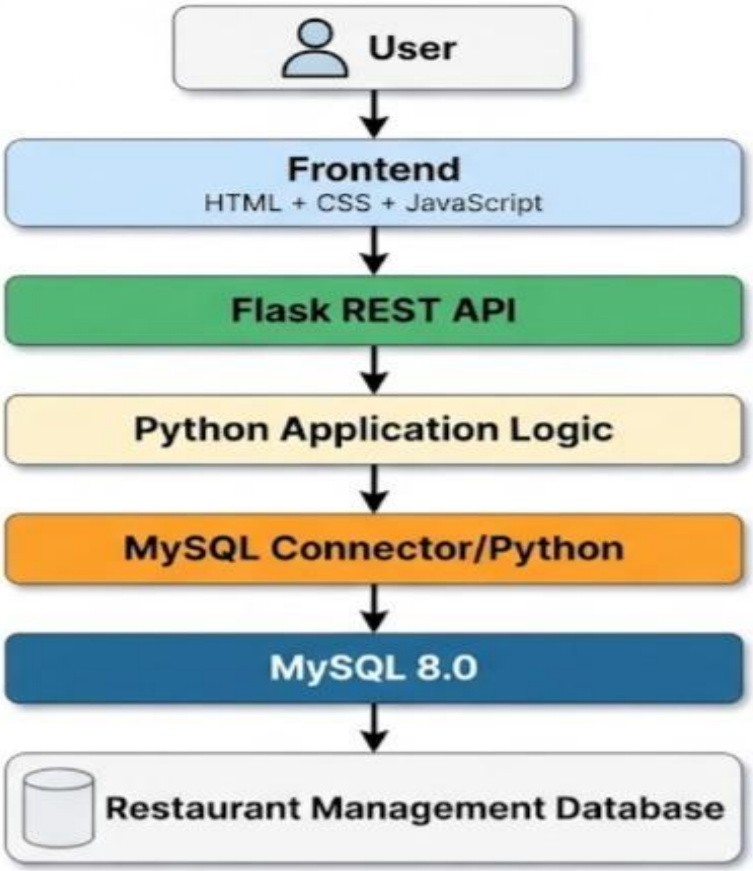
3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming Language	Python
Backend Framework	Flask
Database	MySQL 8.0
Frontend	HTML, CSS, JavaScript
Database Connector	MySQL Connector/Python
IDE	Visual Studio Code
Operating Environment	Windows
Database Name	restaurant_management
Repository	GitHub – Restaurant Management System

Python provides the application programming environment [2], while Flask provides the web application routing layer [3]. MySQL is used for persistent relational storage [1].

Database Connectivity Flow



3.3 Application Modules and Database Mapping

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Display operational summary	Multiple tables/views	Read/Aggregation
Customers	Manage customer information	customer	CRUD
Tables	Manage restaurant tables	dining_area, restaurant_table	CRUD/Status

Reservations	Create and manage reservations	customer, reservation, restaurant_table	Create/Read/Update
Menu	Manage food items	menu_item	CRUD
Orders	Manage customer orders	food_order, order_item, menu_item	CRUD
Kitchen	Track preparation	kitchen_ticket, food_order	Read/Update
Billing	Generate and manage bills	bill, food_order, discount	Create/Read/Update
Payments	Record payments	payment, bill	Create/Read
Reports	Display operational analysis	Reporting views	Read/Aggregation

3.4 CRUD Operations

CRUD represents:

- **Create**
- **Read**
- **Update**
- **Delete**

Table 12. CRUD Operations

Operation	Application Feature / Screen	Tables Affected	Evidence
Create	Customer registration	customer	Customer entry screen
Create	Reservation	reservation	Reservation screen

Create	Order	food_order, order_item	Order screen
Create	Bill	bill	Billing screen
Create	Payment	payment	Payment screen
Read	Dashboard	Multiple tables/views	Dashboard
Read	Customers	customer	Customer listing
Read	Orders	food_order, order_item	Order listing
Read	Reports	Views	Reports screen
Update	Reservation status	reservation	Reservation screen
Update	Order status	food_order	Order/Kitchen screen
Update	Kitchen status	kitchen_ticket	Kitchen workflow
Delete / Controlled Removal	Applicable records	Relevant tables	Restricted by business rules

The application restricts destructive operations where deletion could compromise historical billing or transactional consistency.

3.5 Business Rules, Validation and Error Handling

The application uses validation to prevent invalid business operations. SQL statements should be parameterized so that user input is not directly concatenated into SQL commands. This reduces SQL injection risk and improves query safety.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Expected Behaviour

Guest count	Application/database validation	Guest count must be valid and within table capacity.
Table capacity	Validation trigger/application logic	Reservation rejected if capacity is exceeded.
Reservation time	Validation	End time must be later than start time.
Reservation conflict	Database trigger	Overlapping active reservations are rejected.
Order quantity	Validation	Quantity must be positive.
Monetary values	Validation/check constraints	Invalid monetary values are rejected.
Payment method	Validation	Only supported payment methods are accepted.
Overpayment	Validation trigger/application logic	Payment greater than the bill amount is rejected.
Closed bill	Database triggers	Closed bills cannot be modified or deleted through restricted operations.
Database failure	Exception handling	Failed transactional operations can be rolled back.

Database-Level Validation

The final implementation contains **six validation triggers** supporting important database rules, including reservation conflict validation, guest-count validation, discount validation, payment validation, and protection of closed bills.

No stored procedure is claimed in the final report because the implemented SQL does not contain a stored procedure.

3.6 Reports and Database Operations

The application provides reporting functionality using SQL queries and database views. Reporting views reduce the need for the frontend to repeatedly construct complex aggregation queries.

Table 14. Reports and Database Operations

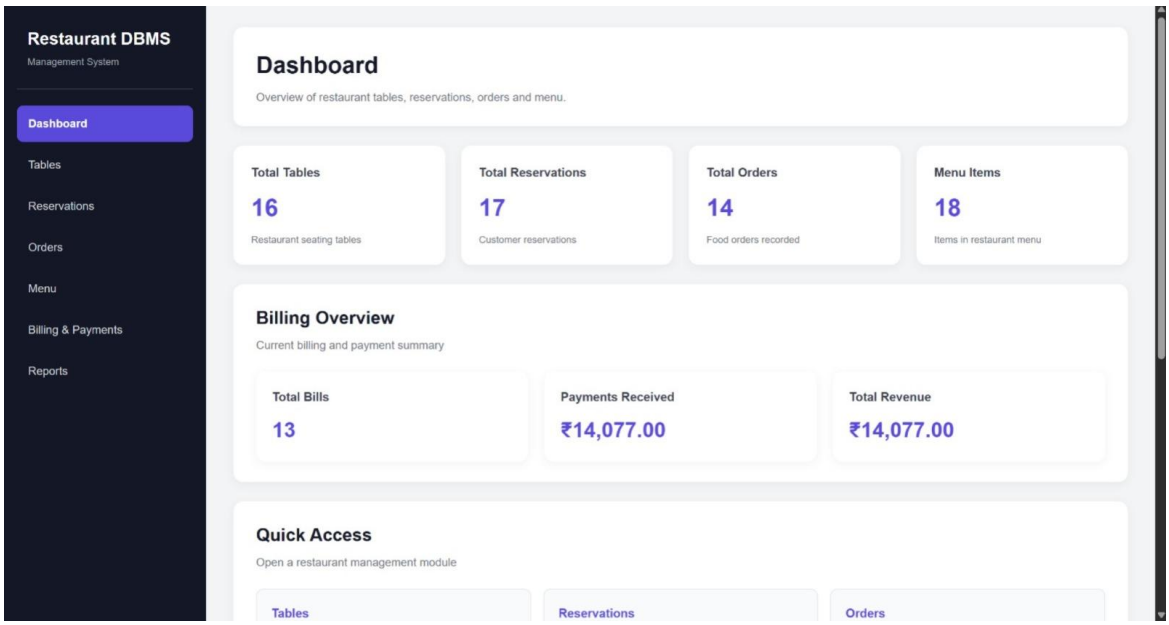
Report / Operation	SQL / Database Concept Used	Application Output
Revenue Summary	Aggregation / reporting view	Revenue information
Item Sales	GROUP BY, aggregation	Item-wise sales
Discount Usage	Aggregation	Discount usage information
Kitchen Delay	Date/time and status analysis	Kitchen performance
Table Turnover	Table/order/billing analysis	Table utilization
Waiter Performance	Aggregation	Waiter-related performance
Operational Overview	Database reporting query/view	Dashboard/report information

The project contains **seven reporting views** used to support operational reporting.

3.7 APPLICATION SCREENS

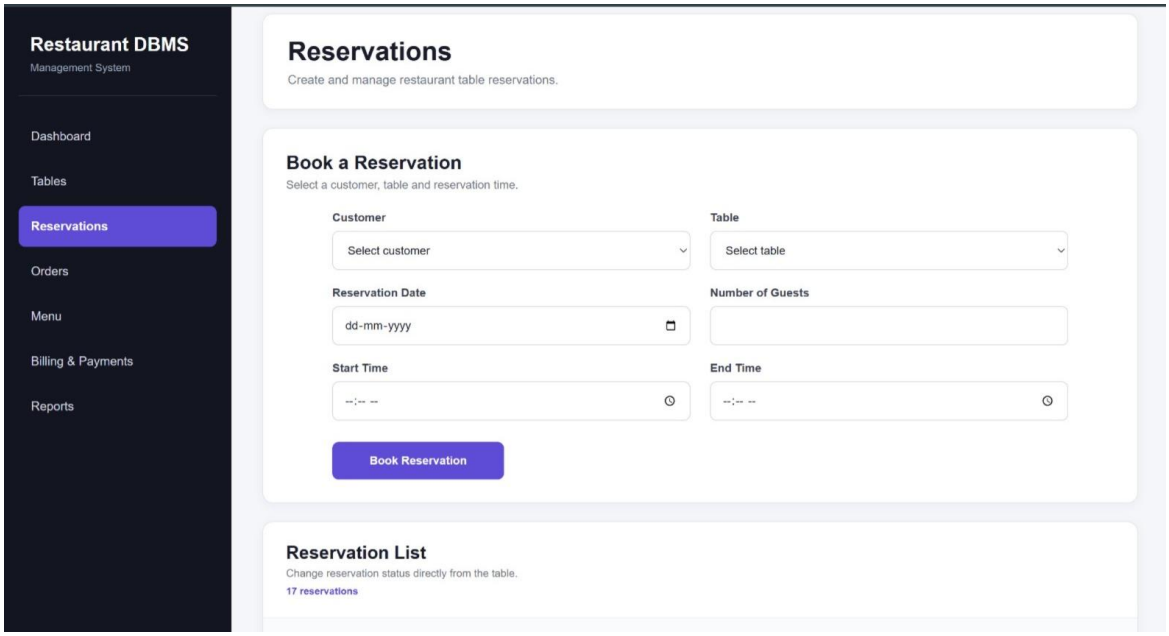
The screenshots should be inserted with captions below each image as required by the report guidelines. The template specifically requires screenshots showing data entry, search/display, update or transaction processing, and at least one report/output. DBMS_PBL_Report_Template_Final_…

Figure 2. Restaurant Management System Dashboard



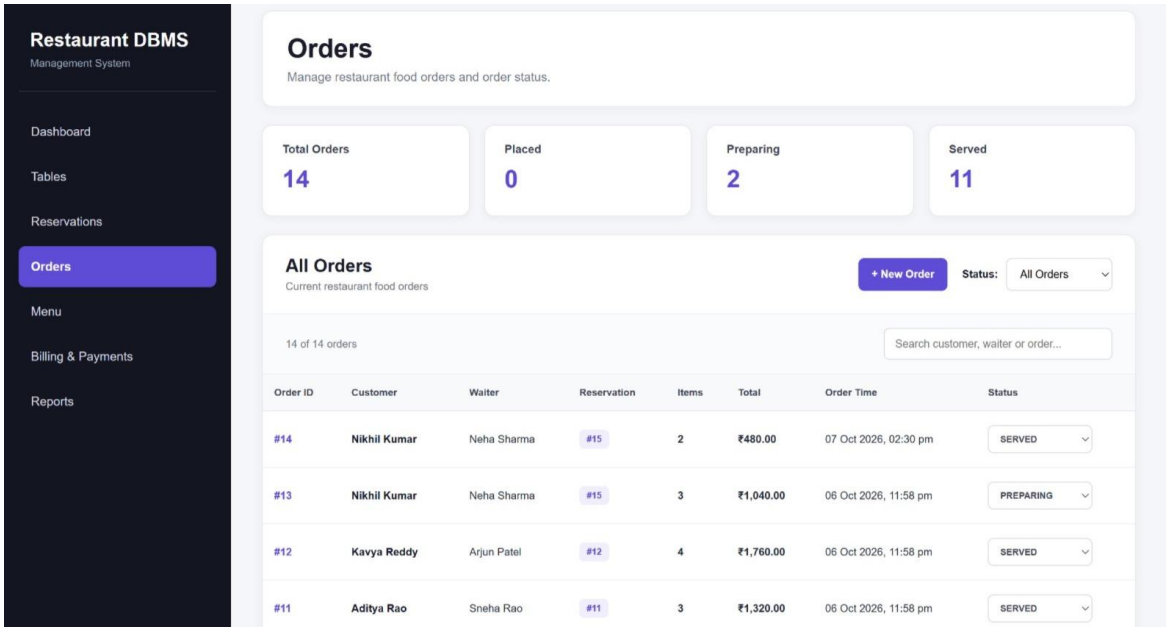
The dashboard provides an overview of restaurant activity, including tables, reservations, orders, bills, payments, and revenue-related information.

Figure 3. Reservation Management Screen



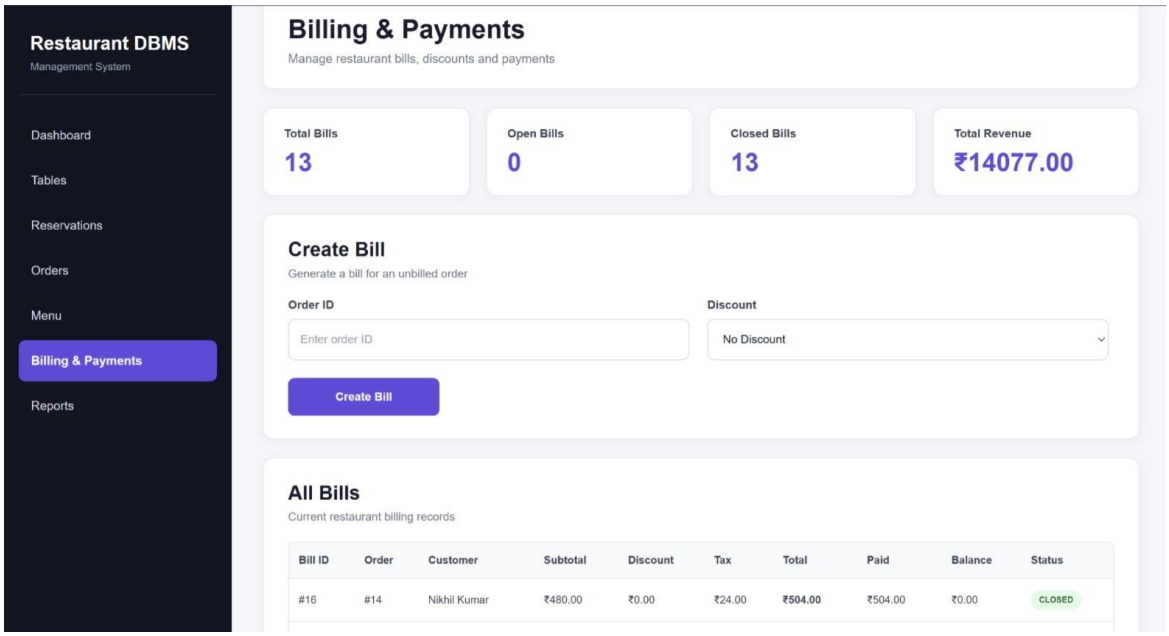
The reservation screen allows restaurant staff to create and manage reservations by selecting customer and table information and entering reservation details.

Figure 4. Order Management Screen



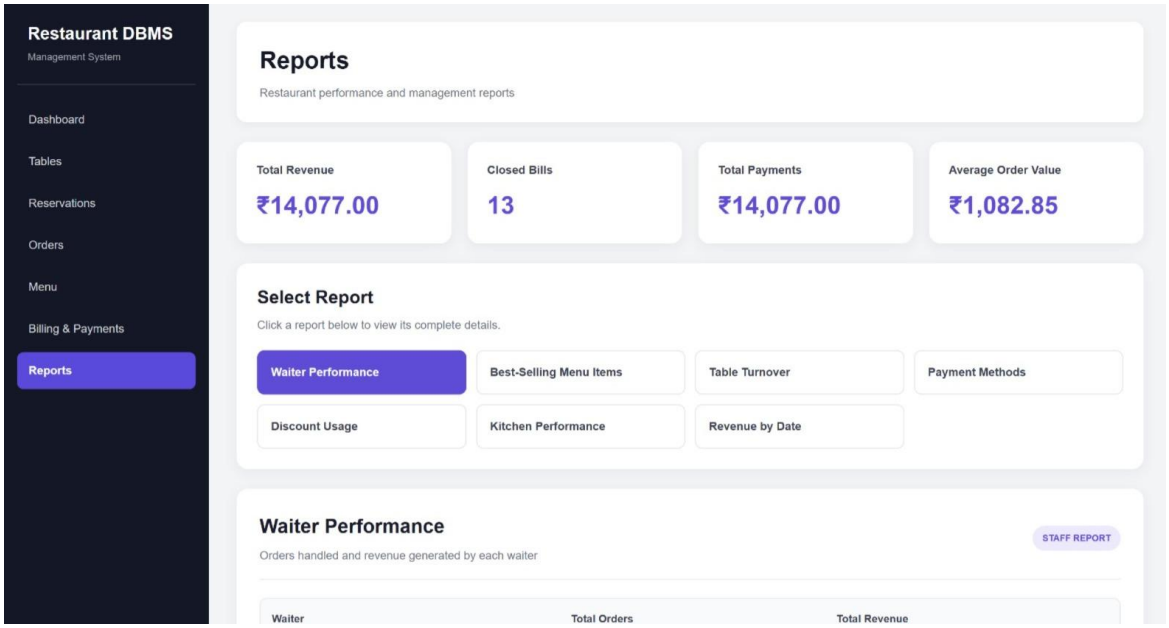
The order screen displays customer orders and their processing status. It demonstrates the connection between orders, menu items, and order-item records.

Figure 5. Billing and Payment Screen



The billing and payment screen displays bill information and allows valid payments to be recorded against bills.

Figure 6. Reports Screen



The reports screen displays operational information generated from database queries and reporting views.

3.8 TESTING AND RESULTS

Testing was performed using meaningful positive and negative scenarios covering reservations, capacity validation, orders, kitchen processing, billing, discounts, payments, and database validation.

The final report should contain **observed results**, not placeholder claims. The submitted template specifically requires expected and actual results to be specific and status to reflect the observed outcome. DBMS_PBL_Report_Template_Final_…

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Reservation	Create valid reservation	Reservation should be created	Reservation workflow	PASS*

				creates a valid reservation	
TC-02	Reservation Conflict	Create overlapping reservation	Request should be rejected	Overlapping reservation is rejected by validation	PASS*
TC-03	Capacity	Enter guests exceeding table capacity	Request should be rejected	Capacity validation rejects the request	PASS*
TC-04	Seating	Update valid reservation/table status	Reservation/table state should update	Valid status update is processed	PASS*
TC-05	Order	Create valid order	Order should be stored	Order and order-item records are created	PASS*
TC-06	Kitchen	Update kitchen ticket status	Ticket status should change	Kitchen ticket status is updated	PASS*
TC-07	Billing	Generate bill for valid order	Correct bill should be generated	Bill is generated for the order	PASS*
TC-08	Discount	Apply valid discount	Valid discount should be reflected	Discount validation is applied	PASS*
TC-09	Overpayment	Pay more than bill amount	Payment should be rejected	Overpayment validation	PASS*

				rejects payment	
TC-10	Payment	Make valid payment	Payment should be recorded	Valid payment is recorded against the bill	PASS*

Important: Replace PASS* with the status observed when you execute the final version. Do not claim a test passed merely because the code appears to support it.

Testing Observation

The test scenarios cover both positive and negative workflows. Positive cases verify that valid reservations, orders, billing operations, discounts, and payments can be processed. Negative cases verify that invalid conditions such as reservation conflicts, capacity violations, and overpayments are rejected. The testing demonstrates that the application interacts with the relational database and applies the important business rules implemented in the system.

CHAPTER 4

CONCLUSION

The Restaurant Management System demonstrates the development of a database-backed web application for managing interconnected restaurant operations. The system converts activities such as customer registration, table management, reservations, menu management, ordering, kitchen processing, billing, discounts, and payments into structured relational data.

The database is implemented using MySQL 8.0 with **12 operational tables**, primary keys, foreign keys, integrity constraints, **seven reporting views**, and **six validation triggers** [1]. Python is used for application logic [2], while Flask provides the backend application and API layer [3]. The frontend provides interfaces for managing restaurant activities and displaying database-backed information.

The project demonstrates important DBMS concepts including ER modelling, relational schema design, functional dependencies, normalization up to 3NF, SQL DDL/DML, CRUD

operations, constraints, validation, reporting, database connectivity, and transaction-oriented application workflows.

Overall, the project provides practical experience in designing and implementing a relational database and integrating it with a web application to solve a realistic restaurant management problem.

4.1 LIMITATIONS AND FUTURE ENHANCEMENTS

Table 16. Limitations and Future Enhancements

Current Limitation	Future Enhancement
Staff authentication is not implemented	Add secure authentication and role-based access control.
External payment gateway is not integrated	Integrate a secure payment gateway.
Real-time kitchen updates are limited	Implement WebSocket or Server-Sent Events for real-time updates.
Reports are limited to implemented reporting operations	Add date-range, peak-hour, revenue trend, and comparative analysis.
Production security configuration is limited	Add HTTPS, stronger CORS configuration, secure secret management, and audit logging.
The database is designed primarily for an academic project environment	Add production backup, monitoring, recovery procedures, and least-privilege database credentials.
Scalability has not been evaluated under production-level load	Introduce performance testing, indexing analysis, caching, and deployment monitoring.

These enhancements would improve the system's suitability for deployment in a larger restaurant environment.

CHAPTER 5

REFERENCES

[1] Oracle Corporation, **"MySQL 8.0 Reference Manual,"** MySQL Documentation. Accessed: Oct. 8, 2026. [Online]. Available: MySQL 8.0 Reference Manual

[2] Python Software Foundation, **"Python 3 Documentation,"** Python Documentation. Accessed: Oct. 8, 2026. [Online]. Available: Python Documentation

[3] Pallets, **"Flask Documentation,"** Flask Documentation. Accessed: Oct. 8, 2026. [Online]. Available: Flask Documentation

CHAPTER 6

APPENDICES

APPENDIX A — COMPLETE DATABASE SCRIPT

https://github.com/nithya-lekhana/Restaurant-Management-System/blob/main/database/restaurant_management.sql

APPENDIX B — APPLICATION SOURCE CODE

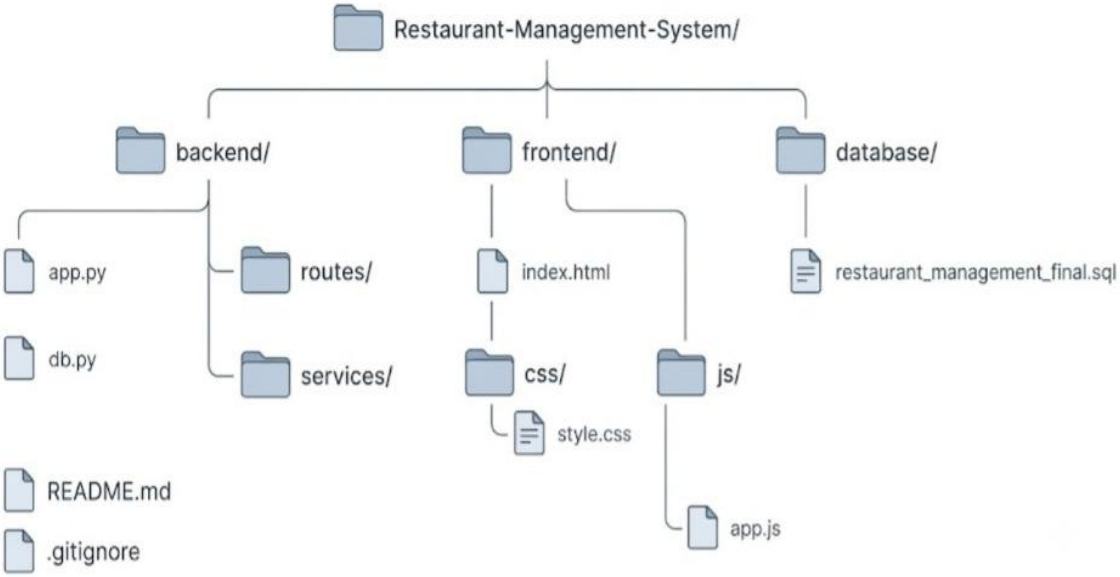
Project Repository

GitHub Repository:

https://github.com/nithya-lekhana/Restaurant-Management-System.git

The repository contains the application source code, backend implementation, frontend files, database script, and supporting project resources.

Major Project Structure



The backend uses Python and Flask to provide application logic and API functionality, while the database layer uses MySQL.

APPENDIX C — APPLICATION SCREENSHOTS

C.1 Dashboard

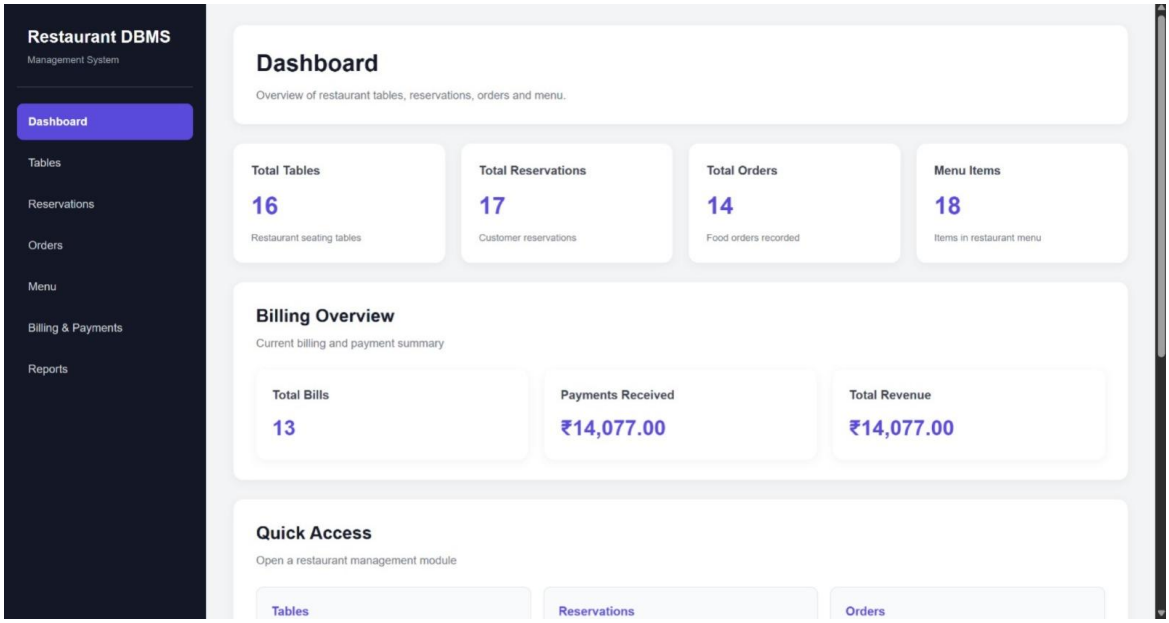


Figure C.1. Restaurant Management System Dashboard

C.2 Reservation Management

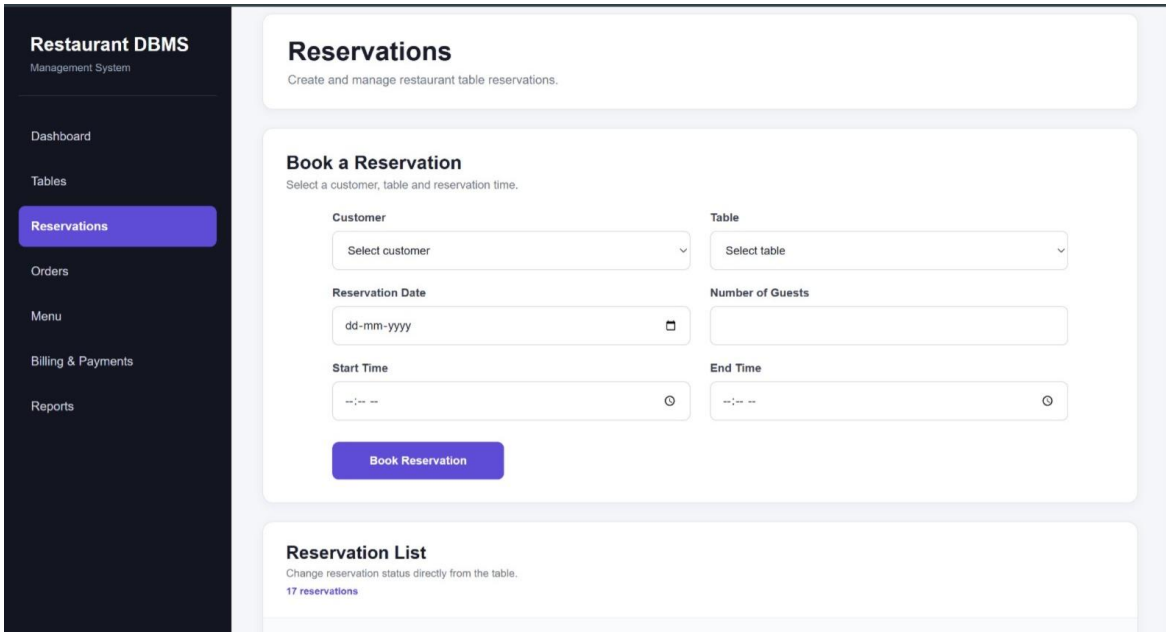


Figure C.2. Reservation Management Screen

C.3 Order Management

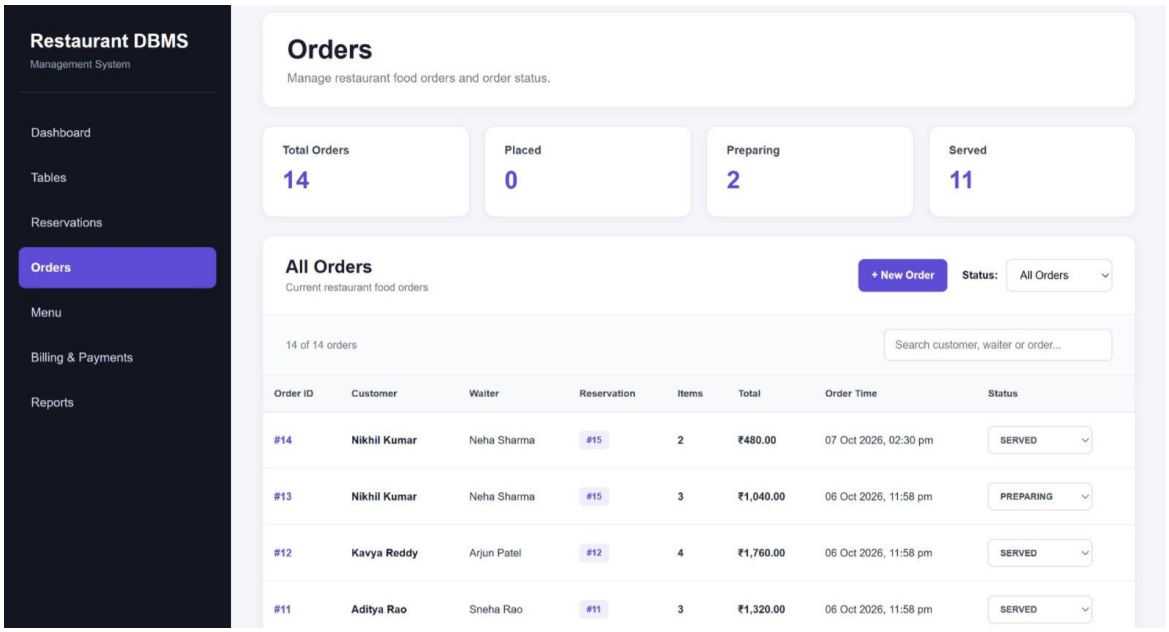


Figure C.3. Order Management Screen

C.4 Billing and Payments

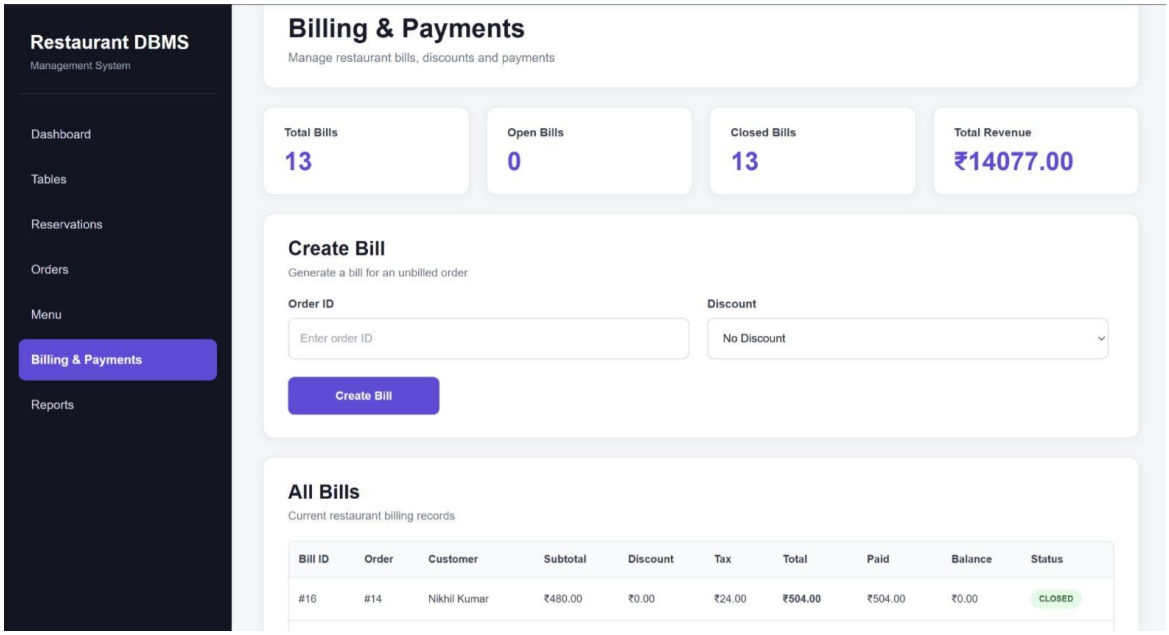


Figure C.4. Billing and Payment Screen

C.5 Reports

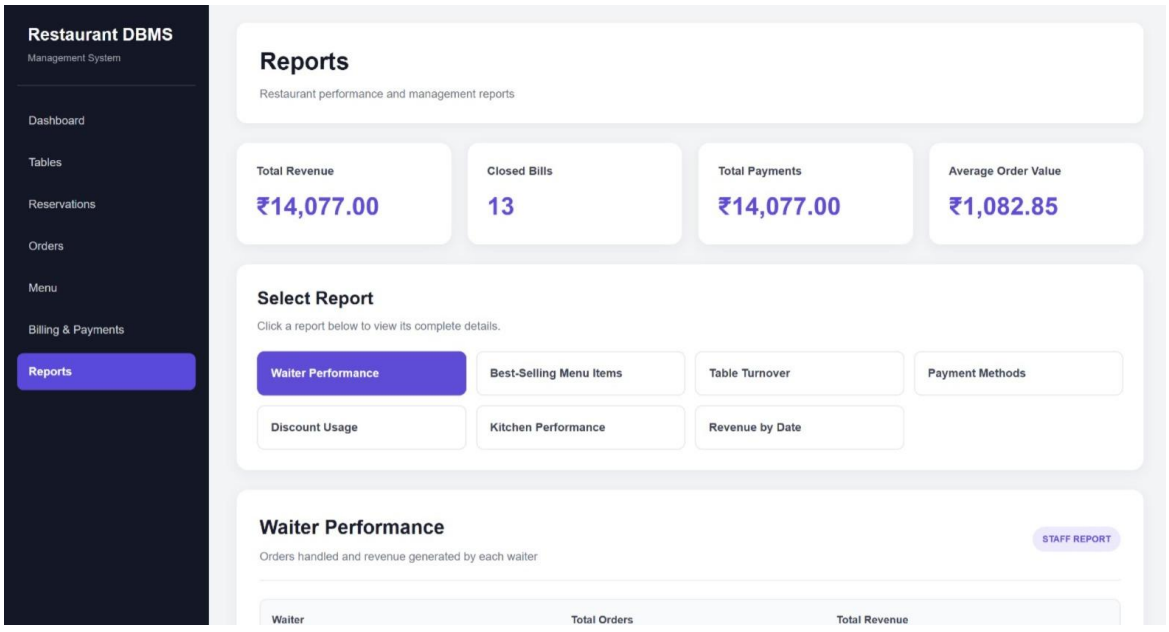


Figure C.5. Reports Screen